

面向游戏玩法的组件

本页面介绍用于将 MIDI 输入与 Unity 游戏逻辑相连接的高级组件。

命名空间：`jp.kshoji.unity.midi`

位置：`Assets/MIDI/Scripts/Gameplay/`

概述

组件	作用
MidiInputMap / MidiInputRouter	由 ScriptableObject 定义的 MIDI 条件 → 触发 UnityEvent
MidiNoteTracker	管理每个通道的按下音符状态
MidiChannelFilter	仅将特定通道转发至下游
MidiDeviceFilter	仅将特定设备转发至下游
SmfPlayer	SMF / Sequence 的同步播放（详见 SMF 工具 ）
MidiRecorder	将实时 MIDI 输入记录为 SMF（详见 SMF 工具 ）
MidiCcSmoother	对 CC 值进行指数移动平均平滑
MidiCcButton	基于 CC 阈值的按钮检测
MidiClockSync	同步外部 MIDI Clock，提供 BPM / 拍 / 小节
MidiClockOutput	作为主控发送 MIDI Clock
SmfPlayerClockAdapter	使 SmfPlayer 的速度跟随外部 Clock
MidiChordDetector	检测和弦名称与音阶归属的变化，判定与目标和弦是否一致（测试）

关于与 Animator / Timeline / Visual Scripting 的集成，请参阅 [Unity 生态系统集成](#)。

以上组件均使用 MidiManager 的事件处理器机制（IMidi*EventHandler）。调用 InitializeMidi() 是场景一侧的职责。

MidInputMap / MidInputRouter

概述

在 `MidInputMap` (`ScriptableObject`) 中定义 MIDI 消息与 `UnityEvent` 之间的映射, `MidInputRouter` (`MonoBehaviour`) 会在条件匹配时触发事件。无需编写代码, 即可从 Inspector 将 MIDI 输入与游戏事件相连接。

设置

- 1. 通过 `Assets > Create > MIDI > Input Map` 创建 `MidInputMap`
- 2. 将 `MidInputRouter` 挂载到 `GameObject` 上
- 3. 将创建好的 Input Map 分配到 `Map` 字段
- 4. 在 Input Map 的 `Bindings` 中设置条件与 `UnityEvent`
- 5. 在场景 bootstrap 中调用 `MidiManager.InitializeMidi()`

绑定条件

字段	说明
<code>messageType</code>	<code>MidiOutgoingMessageType</code> (<code>NoteOn</code> / <code>CC</code> / <code>PitchWheel</code> / <code>Start</code> / <code>SysEx</code> 等)
<code>group</code>	0-15, <code>-1</code> = 所有 group
<code>deviceIdFilter</code>	空 = 所有设备
<code>channel</code>	0-15, <code>-1</code> = 所有通道 (系统 <code>Trigger</code> 中忽略)
<code>noteNumber</code>	0-127, <code>-1</code> = 任意 (<code>Note</code> / <code>Poly Aftertouch</code> / 系统 <code>data byte</code> 过滤)
<code>minVelocity</code> / <code>maxVelocity</code>	<code>NoteOn</code> 的力度范围
<code>controllerNumber</code>	<code>CC</code> 编号
<code>minValue</code> / <code>maxValue</code>	<code>CC</code> / <code>Aftertouch</code> 值的范围
<code>ccTriggerMode</code>	<code>CC</code> 触发方式 (见下表)
<code>programNumber</code>	0-127, <code>-1</code> = 任意 (用于 <code>PC</code>)

消息类型与事件

类别	messageType 示例	触发事件
演奏类	NoteOn/Off, CC, PitchWheel, Aftertouch	onTriggered + onTriggeredWithValue
演奏类	ProgramChange, SongSelect, ...	onTriggered + onTriggeredWithValue (number)
系统 Trigger	TimingClock, Start, Stop, Reset, ...	仅 onTriggered
载荷	SystemExclusive, SystemCommonMessage, Midi2RawUmp	仅 onMessage

MidiInputRouter 本身也具有 raiseMessageEvents / onMessage，可以将所有传入的消息以 DTO（MidiInputMessageEventArgs）形式接收。用于对 SysEx / Raw UMP 的通用处理。

旧版 MidiBindingMessageType 资源会通过 FormerlySerializedAs 自动迁移（PitchBend → PitchWheel）。

CC 触发模式

模式	行为
OnChange	每当值在范围内发生变化时
OnEnterRange	值达到 minValue 及以上的瞬间
OnExitRange	值降至 maxValue 及以下的瞬间
OnThreshold	值达到 minValue 及以上的瞬间（用于按钮）

使用示例

```
// 在 Inspector 中配置 MidiInputMap, 并将游戏逻辑连接到 UnityEvent
// 示例: NoteOn ch0 note60 → 点亮灯光
```

CC 按钮（在 64 及以上时仅触发一次）的配置示例：

- messageType : ControlChange
- channel : 0
- controllerNumber : 64
- minValue : 64
- ccTriggerMode : OnThreshold

注意事项

- 多个绑定按 OR 进行求值（任一匹配即触发）
- 若同时设置了 `onTriggered`（无参数）和 `onTriggeredWithValue`（int 参数），则两者都会触发
- 通过过滤器使用时，请设置 `registerWithMidiManager = false`（[与过滤器组合](#)）

MidiNoteTracker

概述

按 MIDI 通道管理当前处于按下状态（已收到 Note On、尚未收到 Note Off）的音符编号。可作为和弦判定、同时按下数量、按键释放检测等功能的基础。

设置

字段	说明
<code>targetChannels</code>	要追踪的通道（空 = 所有通道）
<code>deviceIdFilter</code>	空 = 所有设备
<code>registerWithMidiManager</code>	是否直接向 <code>MidiManager</code> 注册（默认 true）
<code>onNoteStateChanged</code>	音符状态变化时的 UnityEvent

API

方法	说明
<code>IsNoteOn(channel, note)</code>	指定音符是否处于按下状态
<code>GetActiveNotes(channel)</code>	按下中的音符列表
<code>GetActiveNoteCount(channel)</code>	按下中的音符数量
<code>GetTotalActiveNoteCount()</code>	所有通道的合计
<code>HasAnyNoteOn()</code>	是否有任意音符处于按下状态

在收到 CC 120（All Sound Off）和 CC 123（All Notes Off）时，会将对应通道的所有音符处理为 Off。

使用示例

```
using jp.kshoji.unity.midi;

public sealed class ChordDetector : MonoBehaviour
{
    public MidiNoteTracker tracker;

    void Update()
    {
        if (tracker.GetActiveNoteCount(0) >= 3)
            Debug.Log("3 notes or more pressed on channel 0");
    }
}
```

MidiChannelFilter / MidiDeviceFilter

概述

仅允许特定的 MIDI 通道或设备通过，并将事件转发到下游 GameObject 的过滤器组件。除全部 MIDI 1.0 消息外，还会转发 Raw UMP（IMidi2RawUmpEventHandler）。可用于在 UI 用途与游戏玩法用途之间分离路由等场景。

设置

MidiChannelFilter

字段	说明
allowedChannels	允许的通道（空 = 全部通过）
blockedChannels	屏蔽的通道（优先级高于 allowed）
forwardTargets	转发目标 GameObject 数组

MidiDeviceFilter

字段	说明
allowedDeviceIds	允许的设备 ID（空 = 全部通过）
blockedDeviceIds	屏蔽的设备 ID
allowVirtualDevices	允许带 virtual: 前缀的虚拟设备
forwardTargets	转发目标 GameObject 数组

与过滤器组合

通过过滤器将事件传递给下游组件的接线示例：

```
[MidiManager]
  ↓
[MidiDeviceFilter] ← registerWithMidiManager = true (链的入口)
  ↓ forwardTargets
[MidiChannelFilter] ← registerWithMidiManager = false
  ↓ forwardTargets
[MidiInputRouter]   ← registerWithMidiManager = false
```

重要：使用过滤器链时，请将入口以外的 `MidiChannelFilter` 设为 `registerWithMidiManager = false`，并将下游的 `MidiInputRouter` 和 `MidiNoteTracker` 也设为 `registerWithMidiManager = false`。重复注册会导致事件被重复处理。

工作示例请参阅 `Assets/MIDI/Samples/Gameplay/Scenes/MidiGameplaySampleScene.unity`。

若不通过过滤器而直接使用，则 Router / Tracker 保持 `registerWithMidiManager = true`（默认）即可，不会有问题。

MidiClockSync / 和弦与音阶判定

MidiClockSync

同步来自外部音序器等的 MIDI Clock。BPM、拍、小节事件可从游戏逻辑或 `SmfPlayerClockAdapter` 中使用。

属性 / API	说明
<code>pulsesPerQuarterNote</code>	每拍的 Clock 脉冲数（MIDI 标准为 24）
<code>beatsPerBar</code>	每小节的拍数（默认 4）
<code>deviceIdFilter</code>	空 = 所有设备
<code>EstimatedBpm</code> / <code>IsBpmStable</code>	BPM 估算值与稳定标志
<code>onBeat</code> / <code>onBar</code>	拍 / 小节边界
<code>onStarted</code> / <code>onStopped</code>	收到 Start / Stop

```
var clock = gameObject.AddComponent<MidiClockSync>();
clock.onBeat.AddListener(() => Debug.Log("Beat"));
clock.onBar.AddListener(() => Debug.Log("Bar"));
```

`MidiClockOutput` 可发送 120 BPM 的 Clock。`SmfPlayerClockAdapter` 以 `Follow` / `Step` / `Free` 模式将 `SmfPlayer` 的速度或播放位置与外部 Clock 联动。

和弦与音阶判定

与 `MidiNoteTracker` 组合以进行和弦名称推断与音阶外音符检测。

```
using jp.kshoji.unity.midi.util;

var notes = noteTracker.GetActiveNotes(0);
var chord = ChordRecognition.Recognize(notes);
var inScale = ScaleUtility.AllNotesInScale(notes, rootNote: 0, MidiScaleType.Major);
```

`MidiChordDetector` 通过 `onChordChanged` 通知和弦变化，并通过 `onScaleViolation` 检测音阶外输入。设置 `targetChord` 并调用 `EvaluateNow()` 后，会通过 `onQuizCorrect` / `onQuizIncorrect` 通知与目标和弦的一致性判定（测验）。当按下的音符全部位于音阶内时，`onAllNotesInScale` 会触发。

MidiCcSmoother / MidiCcButton

提供 CC 值的平滑与阈值切换。也可从 Animator 集成等处使用。

MidiCcProcessorBase

CC 处理器组件的抽象基类。提供通道 / 设备匹配辅助，以及 EMA / 基于时间的平滑辅助（EMA 委托给 `MidiControlSmoothing`）。

位置：`Assets/MIDI/Scripts/Gameplay/MidiCcProcessorBase.cs`

API	说明
<code>MatchesChannel / MatchesDevice</code>	过滤器辅助（-1 / 空 = 全部匹配）
<code>ApplyEma(...)</code>	委托给 <code>MidiControlSmoothing.ApplyEma</code>
<code>SmoothTowards(current, target, smoothingTime, deltaTime)</code>	朝归一化目标进行基于帧的 lerp

MidiCcSmoother

项目	说明
<code>configs[]</code>	控制器编号、通道、 <code>smoothFactor</code> （0.01–1.0，越小越平滑）
<code>onSmoothedValue[]</code>	归一化值 (0–1) 的 UnityEvent
<code>GetSmoothedNormalizedValue(index)</code>	获取最新的归一化值

MidiCcButton

项目	说明
<code>onThreshold / offThreshold</code>	带迟滞的阈值（默认 64 / 63）
<code>onPressed / onReleased</code>	按下 / 释放事件
<code>IsPressed</code>	当前的开启状态

源文件

文件	作用
MidiInputMap.cs	ScriptableObject 定义
MidiInputBinding.cs	绑定定义
MidiInputCondition.cs	条件求值逻辑
MidiInputBindingMigration.cs	旧 enum 迁移
MidiInputMessageEventArgs.cs	onMessage DTO
MidiInputRouter.cs	事件路由器
MidiNoteTracker.cs	音符状态管理
MidiNoteStateChangedEvent.cs	状态变化事件参数
MidiFilterBase.cs	过滤器基类
MidiChannelFilter.cs	通道过滤器
MidiDeviceFilter.cs	设备过滤器
MidiEventForwarder.cs	事件转发辅助
SmfPlayer.cs / SmfPlayerState.cs	SMF 播放
MidiRecorder.cs	SMF 录制
MidiClockSync.cs / MidiClockState.cs / MidiClockOutput.cs	MIDI Clock 同步
SmfPlayerClockAdapter.cs	SMF 与外部 Clock 的速度联动
MidiChordDetector.cs	和弦与音阶判定（实时检测 + 测验）
MidiCcProcessorBase.cs	CC 处理器的共享基类
MidiCcSmoother.cs / MidiCcButton.cs	CC 平滑与阈值切换
Gameplay/Theory/ChordRecognition.cs / Gameplay/Theory/ScaleUtility.cs	和弦名称推断与音阶工具
MidiManagerReceiverBridge.cs	Sequencer → MidiManager 桥接

相关文档

- [MIDI 1.0 运行时 \(MidiManager\)](#)
- [工具类 \(MidiNoteUtility / MidiMessageBuilder\)](#)
- [编辑器工具 \(MIDI 监视器\)](#)
- [SMF 工具 \(SmfPlayer / MidiRecorder\)](#)
- [入门指南](#)